

Smart Electronics Lab v1.4

A three-kit product ladder with a no-code default path: guided electronics lab,
internet radio, and SDR workstation

Prepared for Bob Litt

Revised June 2026 — No-Code User Paths & Firmware Delivery Architecture

v1.4 · supersedes v1.3

0. What changed in v1.4

v1.4 adds a no-code product architecture on top of v1.3's three-kit ladder. The design problem: none of the three kits should require the ordinary learner to write or compile code to reach their main milestones. Coding remains possible and teachable, but it moves to an explicit, optional Developer Mode path (§20) rather than being the default way to get a kit working. v1.4 changes nothing about the kit ladder, BOM, hardware design, or lesson content decided in v1.3 — it adds the user-facing and firmware-delivery layer that sits on top of all three kits.

Change	What happened
Hard product rule	A learner must be able to complete Kit 1, Kit 2, and Kit 3's main milestones without writing code. See §20.
Three user paths	Use Mode (default, no-code, prebuilt firmware), Configure Mode (no-code, editable settings), and Developer Mode (PlatformIO/Arduino, optional, never required for a kit milestone). See §20.
Firmware delivery architecture	Three firmware images defined: Elecrow Tutor Firmware, ESP32 Audio Firmware, and a later SDR Bridge mode of the same Elecrow firmware — each with explicit responsibilities and a JSON command/status protocol. See §21.
Browser-based flashing	A web installer (ESP Web Tools / ESP Launchpad class tool) is now the default Use Mode flashing path; Arduino IDE/PlatformIO are not required for normal use. Flagged that this needs a desktop browser with Web Serial — not assumed to work on iPhone/iPad. See §22.
Repository structure	Proposed a concrete GitHub repo layout (firmware/, shared-protocol/, lesson-packs/, installers/, docs/, hardware/) and a release-artifact checklist. See §23.
First-time user flow	Spelled out as product behavior, not implementation: Kit 1 Part 1 first boot, Kit 1 Part 2 radio capstone, and Kit 2/3 first boot. See §24.
Configuration schema	Added a minimal Kit 1 config schema (Wi-Fi, stations, lesson pack, calibration, safety) covering everything Configure Mode needs to expose without coding. See §25.
Lesson schema extension	Extended §7's lesson schema with firmware/hardware requirement fields, needed for a no-code lesson engine to self-describe its own requirements before a lesson starts. See §26.
Product branch vs. developer branch	Formalized as a hard rule: every required kit milestone must be achievable on the Product branch (prebuilt firmware) alone. See §27.
Build plan for Claude/Codex	Added a concrete build-priority order and three explicit MVPs (LED/resistor lesson, radio playback, SDR knob control), restated for the implementers rather than the buyer. See §28.

Support, recovery, flash behavior	Added recovery/self-test requirements and restated v1.2/v1.3's flash-write and storage decisions (§7) under the no-code framing. See §29.
Kit 2/3 image strategy	Extended §13 with a no-code requirement: Kit 2/3 users get a prepared Pi/mini PC image or packaged installer, not a compile-from-source path. See §30.
Product identity statement	Added a one-paragraph product identity statement: this is a guided electronics learning system with an optional developer track, not a coding kit. See §31.

Nothing in this brief contradicted a v1.3 decision; it adds a layer above the existing kit/hardware/lesson design rather than changing it. The one judgment call worth flagging: Firmware Image 3 (SDR Bridge, §21) is documented as additional modes of the existing Elecrow tutor firmware rather than a fourth separate codebase, since Kit 2/3 already reuse the same physical Elecrow unit carried over from Kit 1 (§13).

1. Decision log UPDATED

Accepted

- Elecrow remains tutor and control head across all three kits.
- Elecrow is **not** assumed to have a usable audio output. Kit 1's internet-radio capstone always uses a second, dedicated ESP32 audio module.
- Kit 1 culminates in an internet-radio "sound moment" — this is the capstone, not an optional side feature.
- The audio module stays in the ESP32/Arduino world: ESP32-S3 dev board + MAX98357A breakout on the breadboard (§6), not the integrated ESP32-A1S/ESP-ADF path.
- SDR/waterfall capability moves entirely out of Kit 1, into Kit 2 (add-on, user's monitor) and Kit 3 (Kit 2 + included 15" display).
- Raspberry Pi / mini PC compute is reserved for Kit 2/3 only — never introduced just for internet radio.
- RTL-SDR Blog V4 is the default Kit 2/3 receiver recommendation (native HF,

Open

- Exact second-ESP32 audio board SKU/vendor listing (direction set in §6; not yet sourced to a specific listing).
- Exact amp/speaker pairing and whether a small enclosure/speaker box is needed.
- Elecrow-to-audio-module physical link: UART recommended as the simplest default; I2C and Wi-Fi (e.g. ESP-NOW) remain documented alternatives.
- Kit 2 compute: Raspberry Pi 5 vs. Pi 4 vs. x86 mini PC. 2026 DRAM pricing has pushed Pi 5 and mini PCs close to cost parity, so this is a genuine trade-off now, not a default (§13).
- Whether Kit 2 ships purely as a Kit-1-dependent add-on, or also as a partially standalone advanced kit for someone who skipped Kit 1.
- Kit 3 touch vs. non-touch 15" display — default assumption is non-touch, since the Elecrow already provides physical control (§14).

better filtering than V3, current community default, ~\$35–40).

- SDR++ is the default first-success waterfall software for Kit 2/3 (modern, cross-platform, lighter than GQRX on a Pi).
- USB-serial JSON-per-line (carried from v1.1/v1.2) generalizes to audio and SDR commands, not just knob/touch events (§6, §13).
- First product path remains standard breadboard + smart power/measurement + display tutor — never a proprietary smart breadboard.
- 3.3V default rail, INA219 topology, extended lesson schema, structural power-off-before-rewiring, and the controlled fault-screen drill all carry forward from v1.2 unchanged.
- Kit 1 ships in two customer-facing tiers: **Kit 1 Standard** (Part 1 only) and **Kit 1 Deluxe** (Part 1 + Part 2, adding the second ESP32 audio module, I2S DAC/amp, speaker, and the internet-radio lesson pack). The tier split is a deliberate product and curriculum decision, not only a workaround for the Elecrow's unconfirmed audio path.
- A future v2 custom dial/display module is documented as a direction (§19), converging the Elecrow's and the second ESP32 audio module's functions into one device — explicitly deferred until after the MVP is proven, and explicitly not a blocker or delay for the v1.3 plan.
- Default user path is no-code (§20); coding is an optional Developer Mode branch, not a prerequisite, for any of the three kits.
- Firmware is prebuilt for Use Mode; browser-based flashing (ESP Web Tools/ESP Launchpad class tool) is the preferred installer, not Arduino IDE/PlatformIO (§22).

- Elecrow expansion connector type and exact pinout (carried from v1.2, still unconfirmed).
- Elecrow I2S/audio feasibility — now a low-stakes curiosity rather than a Kit 1 blocker, but still worth checking once for future flexibility.
- ESP Web Tools vs. ESP Launchpad as the primary installer (§22).
- Exact firmware packaging/release format (§23).
- Exact station-configuration interface — on-device menu vs. local web UI vs. config file (§20, §25).
- Whether the Kit 2/3 Pi image is custom-built or assembled via install scripts on stock Raspberry Pi OS (§30).
- How much lesson content lives in firmware vs. filesystem for v1 (§7, §29).
- Whether a local web UI is needed in v1 or can wait until v2 (§20).

- Configure Mode is allowed and documented (§20, §25), but it is configuration, not coding.
- Developer Mode exists and is documented (§20, §23), but is optional for every kit milestone.
- Kit 1 Part 2 must produce the sound moment without requiring code editing (§20, §24).
- Kit 2/Kit 3 use a prepared SDR software path — an image or installer — not a compile-from-source path (§30).
- Claude/Codex build the first firmware; users install it through the Product branch (§27, §28).
- Firmware Image 3 (SDR Bridge) is additional modes of the existing Elecrow tutor firmware, not a separate codebase (§21).

2. The MVP, stated plainly NEW

The first MVP is **not** SDR. It is **not** a 15-inch screen. It is **not** a proprietary smart breadboard.

The first MVP is **one guided LED/resistor lesson** where the Elecrow tutor reads measured current from the INA219 and responds correctly — a pass message on expected current, a fail message on an open circuit or an out-of-range reading.

Everything else in this document is what gets built *after* that one lesson works end-to-end.

3. Three-kit product ladder

The distinctive idea is not "an electronics kit." It is a guided breadboard system where the device **teaches, observes, and reacts**. The Elecrow rotary display is the tutor throughout: it shows the lesson, accepts knob/touch input, reads smart measurement data, and guides the learner step by step — across all three kits, not just the first.

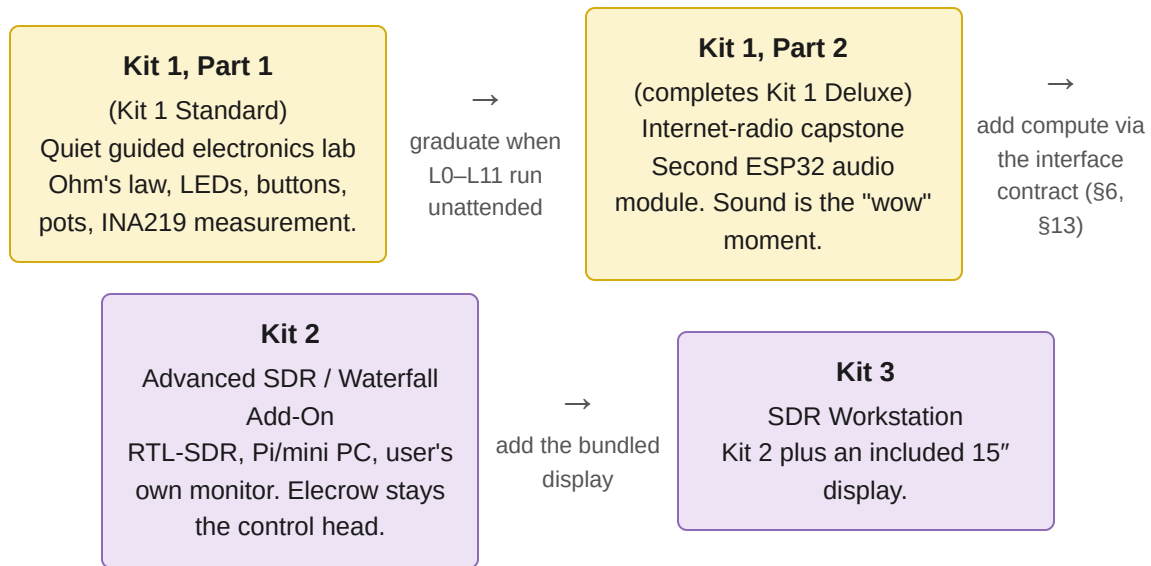


Figure 1. The three-kit product ladder. Kit 1 is one purchase with two parts; Kit 2 and Kit 3 are separate, optional purchases for users who want to go further.

Product tiers vs. build sequence: "Part 1" / "Part 2" above describe build sequence; **Kit 1 Standard** / **Kit 1 Deluxe** are the customer-facing names for the same boundary. Kit 1 Standard is Part 1 only. Kit 1 Deluxe is Part 1 plus Part 2 — the second ESP32 audio module, I2S DAC/amp, speaker, and the internet-radio lesson pack. The split is a deliberate product and curriculum feature, not only a workaround for the Elecrow's unconfirmed audio path (§6).

Kit	Purpose	New compute?	Display	Exit criteria
Kit 1, Part 1 (Kit 1 Standard)	Quiet guided electronics lab	No — Elecrow only	Elecrow 2.1"	A beginner completes L0–L11 using only the on-screen tutor, with at least one correct INA219 current/voltage validation.
Kit 1, Part 2 (completes Kit 1 Deluxe)	Internet-radio capstone	Second ESP32 (audio only)	Elecrow 2.1"	The learner selects a saved station on the Elecrow and the second ESP32 audio module plays internet-radio audio through a speaker.
Kit 2	Advanced SDR / waterfall add-on	Raspberry Pi or mini PC	User-supplied monitor	The Elecrow knob changes an SDR parameter (tuned frequency, waterfall zoom) on the Pi/mini PC, visible on the user's monitor.
Kit 3	SDR workstation, all-in-one	Same as Kit 2	Included 15" display	Same as Kit 2, but the waterfall renders on the included 15" display.

Design rule (unchanged): every early purchase remains useful later. The breadboard teaches and prototypes. It does not become the final enclosure. The Elecrow purchased for Kit 1 is the same unit used as the control head in Kit 2 and Kit 3.

4. Open risks

None of these block ordering the Kit 1 parts list. They block committing scope for the kit they're listed under.

Risk	Why it matters	Default assumption	Resolved by
Elecrow connector/pinout unconfirmed	Determines INA219 wiring and whether an adapter cable is needed	Qwiic/STEMMA-compatible, unverified	Photograph + identify on first-boot (§17)
Elecrow I2S/audio feasibility	Was a Kit 1 blocker in v1.2; now just a future-flexibility question	Not available; not needed since Kit 1 audio moved off-Elecrow	Optional check, no deadline
Exact audio module SKU	Direction (ESP32-S3 + MAX98357A) is set; specific listing is not	Any ESP32-S3 dev board + MAX98357A breakout pair	Sourcing pass before Kit 1 Part 2 build
Elecrow ↔ audio-module link	UART, I2C, and Wi-Fi are all technically possible	UART (simplest hardware, no addressing)	First Kit 1 Part 2 build attempt
Kit 2 compute choice	Pi 5 and mini PCs are near cost parity in 2026; tradeoffs differ (ARM ecosystem vs. x86 driver coverage)	Raspberry Pi 5 (ecosystem/documentation fit); mini PC if Pi price/stock is bad	§13 research; final call at Kit 2 purchase time
Kit 2 standalone vs. add-on	Affects whether Kit 2 needs its own onboarding/UI path	Add-on only, assumes Kit 1's Elecrow	Revisit if a no-Elecrow Kit 2 SKU is requested
Kit 3 touch display	Touch adds cost and complexity	Non-touch; Elecrow is the physical control	Revisit only if user testing shows a real gap

5. Kit 1, Part 1: hardware design

3.3V logic warning: the Elecrow / ESP32-S3 GPIO and I2C pins are 3.3V logic. Do not connect 5V directly to any GPIO, I2C, UART, touch, display, or expansion pin unless the Elecrow documentation explicitly states that pin is 5V tolerant. The breadboard may use 5V for isolated LED/resistor experiments that never touch an Elecrow pin — but any circuit connected to Elecrow data pins defaults to 3.3V logic.

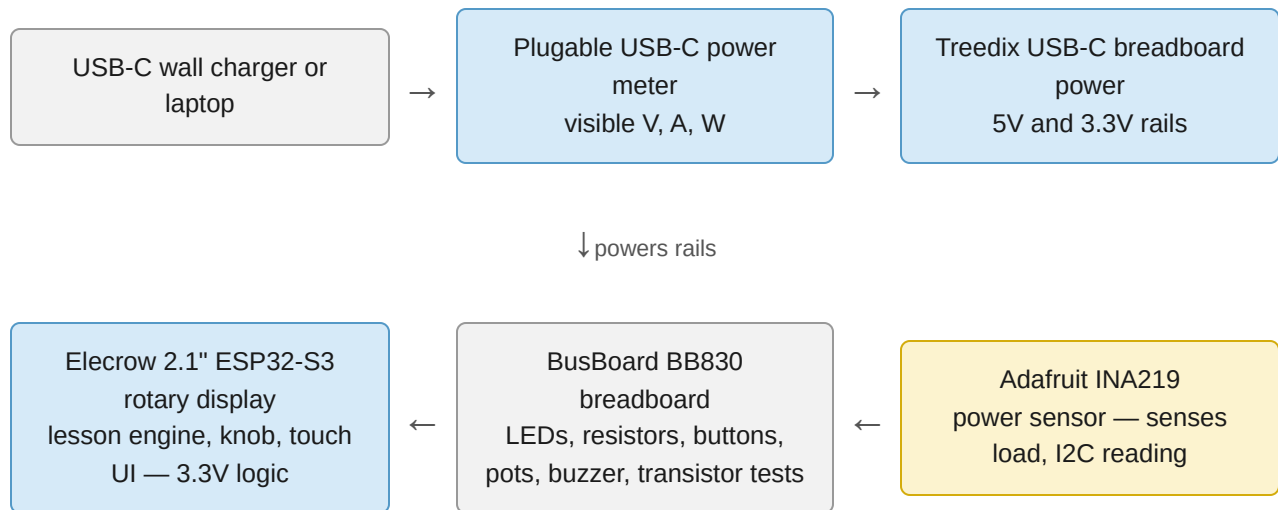


Figure 2. Kit 1, Part 1 hardware architecture (unchanged from v1.1/v1.2).

Version 0 is human-observed with the USB-C meter. Version 1 is software-observed with the INA219. Version 2 becomes a custom Smart Power Module.

Mounting answer

The Elecrow display does not need to sit in the breadboard. It sits next to the BB830 on the desk or mat, then connects to the breadboard through its GPIO/I2C connector or pigtail. The breadboard is for temporary circuits. The display is the tutor and controller.

Smart board answer

Do not start by designing a proprietary smart breadboard. Start by making the power path smart. If the Elecrow can read voltage and current from an INA219 sensor over I2C, the lesson engine can compare expected and measured behavior.

INA219 measurement topology

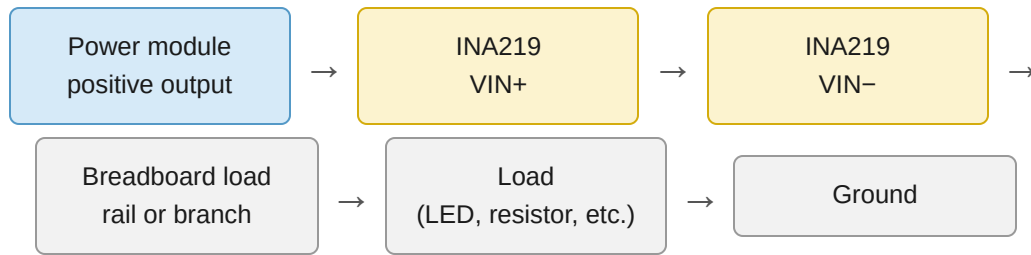


Figure 3. Correct INA219 placement — in series with the measured load, not in parallel with it. (Unchanged from v1.2.)

- The INA219 must be in the measured load's current path. If the load can draw current through any path that bypasses the INA219, the software cannot detect that current.
- The Elecrow, INA219, and breadboard circuit must share a common ground.
- The INA219's I2C logic side should be powered at the Elecrow-safe logic voltage — normally 3.3V — unless the Elecrow's I2C pins are confirmed 5V tolerant.

6. Kit 1, Part 2: internet-radio capstone NEW

Kit 1's "wow" moment is the first time the learner hears sound from something they assembled. This section assumes the Elecrow does **not** expose a clean, confirmed audio path, and routes the radio capstone through a second, dedicated ESP32 audio module instead — so the capstone never depends on an unconfirmed Elecrow capability.

Why a second module, even if Elecrow audio turns out fine: the second ESP32 audio module is not only a workaround for the Elecrow's uncertain audio path. Even if the Elecrow turned out to support clean audio output, Kit 1 Deluxe would likely still keep audio on a separate module: it keeps the tutor/control-head firmware free of streaming and decode workload, and it turns the radio capstone into a distinct, sellable tier (Kit 1 Deluxe over Kit 1 Standard) rather than a feature silently folded into the base unit.

Researched options for the audio subsystem

Option	Approx. cost	Beginner fit	Verdict
Generic ESP32 dev board + MAX98357A breakout	\$5–8 + \$4–6	High — huge tutorial base, Arduino-native	Workable, but less headroom than -S3 for future use
ESP32-S3 dev board + MAX98357A breakout	\$8–12 + \$4–6	High — same wiring as above, more PSRAM/headroom	Recommended

ESP32-A1S Audio Kit (Ai-Thinker, integrated codec)	\$10–15	Lower — less wiring, but expects ESP-ADF, which beginners consistently report as harder than the Arduino framework	Documented alternative, not the default
M5Stack Atom Echo / Core2 (integrated speaker+mic+amp)	\$13–30+	Lowest wiring burden, but appliance-like — little left to breadboard	Skips the pedagogy this kit is built around

Recommendation: an ESP32-S3 dev board plus a MAX98357A I2S amplifier breakout, wired on the breadboard, programmed with the Arduino framework (the ESP32 - audioI2S library by schreibfaul1 is the most widely used library for exactly this). Reasoning against the brief's evaluation criteria: it is the cheapest path that stays fully in the ESP32/Arduino ecosystem already used for the Elecrow; it has the deepest base of existing internet-radio tutorials and example code; the MAX98357A needs only three signal pins (BCLK, LRC, DIN — no MCLK), which keeps the wiring beginner-friendly and keeps the build on the breadboard, consistent with Kit 1 Part 1's pedagogy; and it avoids ESP-ADF, which both beginners and experienced makers report as harder to configure than the integrated ESP32-A1S board's wiring savings are worth. The ESP32-A1S and M5Stack paths remain valid documented alternatives for someone who wants less wiring at the cost of either a steeper firmware learning curve (A1S/ESP-ADF) or a more appliance-like, less breadboard-centric build (M5Stack).

Architecture

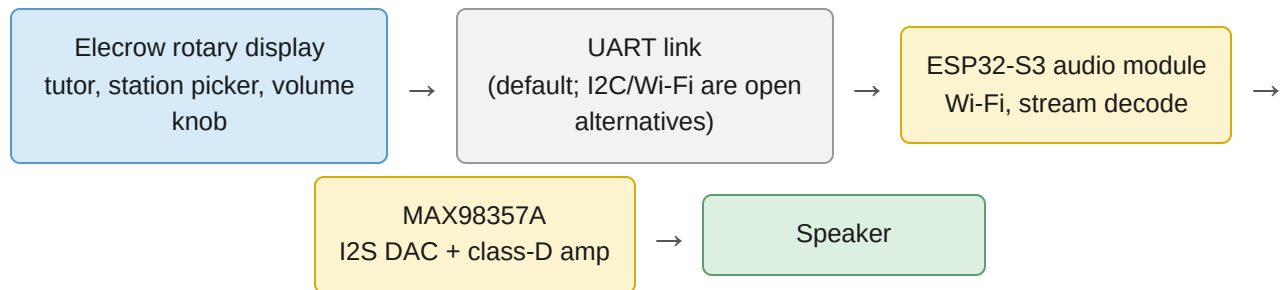


Figure 4. Kit 1, Part 2 architecture. The audio module owns Wi-Fi streaming and decoding; the Elecrow stays the tutor and the only thing the learner directly operates.

Command protocol, generalized EXTENDED

The v1.1/v1.2 USB-serial JSON-per-line direction carries forward, generalized to cover audio commands (and, in §13, SDR commands) alongside the original knob/touch/button events:

```

{"event": "knob", "delta": 1}
{"event": "button", "name": "select", "state": "pressed"}
{"event": "touch", "x": 214, "y": 93}
{"command": "play_station", "id": "station_001"}

```

```
{ "command": "set_volume", "value": 42 }  
{ "status_request": "audio" }
```

The Elecrow's firmware should already emit knob/touch/button events in this shape during Kit 1 Part 1, before the audio module exists to consume them. That is what avoids a rewrite when Part 2 is added.

What Part 2 adds to the lesson content

v1.1/v1.2's "Stage 2a" curriculum (Wi-Fi association, fetching a JSON API) did not get a separate slot in the new kit ladder — it is folded directly into the Part 2 lesson sequence (L12–L13, §11), since connecting to Wi-Fi and fetching a station list is exactly what the radio capstone already requires.

7. Software loading and lesson content

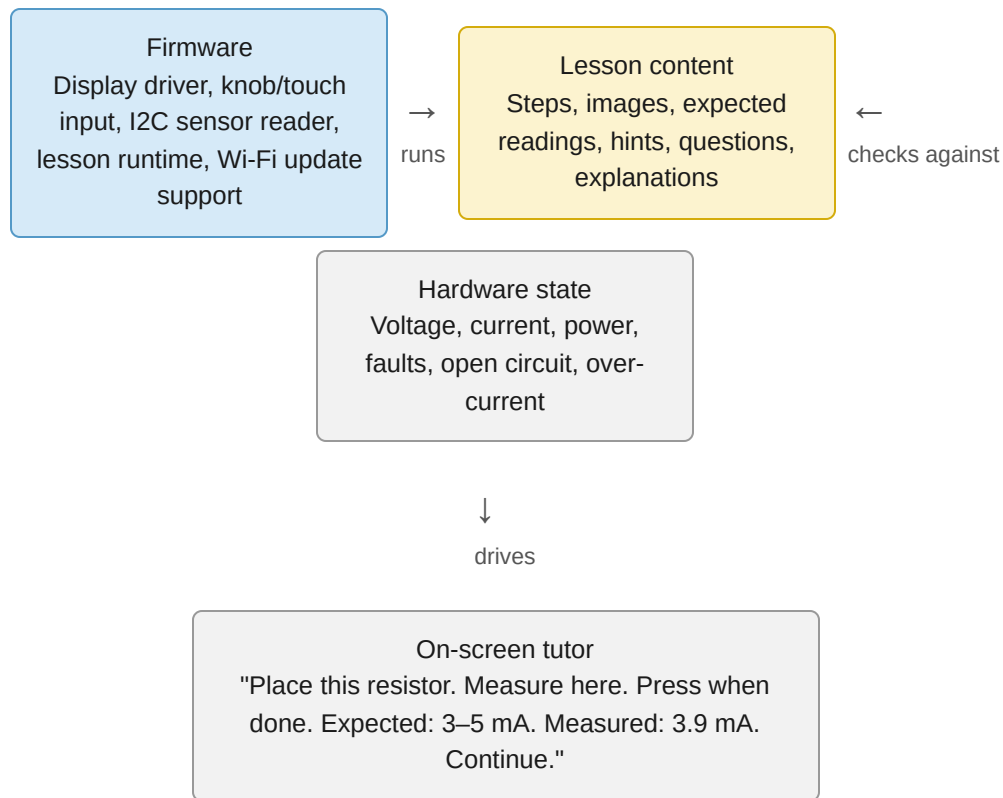


Figure 5. Software architecture for the tutor and lesson packs. (Unchanged from v1.2.)

Lesson schema

Unchanged from v1.2 — extended with safety and measurement-precision fields so assumptions are data, not prose, and with a structural power-off-before-rewiring behavior:

```

id: L2_led_with_resistor
title: "LED with resistor"
supply_rail: 3v3
steps:
  - prompt: "Place a 330 ohm resistor from row A10 to A15."
    wait_for: knob_press
    requires_power_off_before_step: true
  - prompt: "Place the LED long leg at A15, short leg to the ground rail."
    wait_for: knob_press
    requires_power_off_before_step: true
  - prompt: "Turn power on."
    measurement_source: ina219
    expected_units: mA
    expected_range: [3, 5]
    warning_threshold: 6
    hard_fault_threshold: 25
    max_safe_current_mA: 25
    gpio_safe: true
    requires_supervision: false
    on_pass: { advance: true, message: "Correct. Press to continue." }
    on_fail:
      open_circuit: "No current detected. Check the ground rail connection."
      over_range: "Current is higher than expected. Check resistor value and LED polarity"
hints:
  - "LED current is set mostly by the resistor, not the LED."
  - "Red or yellow LEDs work best at 3.3V; blue/green/white need more headroom."
next: L3_change_the_resistor

```

Any step with `requires_power_off_before_step: true` makes the engine render a standard interstitial — "Disconnect power before changing this wiring. Press when power is disconnected." — before the step prompt. `measurement_source` distinguishes a human-read value (`user_meter`, `usb_meter`) from a software-read one (`ina219`, `gpio_adc`). `warning_threshold` / `hard_fault_threshold` / `max_safe_current_mA` separate "this looks off" from "this is dangerous" from "this lesson's safe ceiling," and are what L10's controlled fault drill manipulates without ever exceeding the circuit's real safe operating current.

Developer mode

Connect the Elecrow to a Mac or PC with a USB-C data cable. Use PlatformIO first if possible, or Arduino IDE if following Elecrow examples. Upload firmware over USB-C. The Kit 1 Part 2 audio module is developed the same way, as a second, independent Arduino/PlatformIO project.

User mode

Later, the device should update over Wi-Fi. A user would choose a lesson pack from a menu and the device would download content. This is not Kit 1 work.

Storage plan

Do not buy a microSD card yet. The Elecrow listing specifies 16M flash, UART, I2C, and FPC expansion, but does not advertise a microSD slot. The v1 plan:

- Firmware uploaded over USB-C during development.
- Lesson content (the schema above) compiled into firmware or stored in internal flash using LittleFS or an equivalent ESP32 filesystem.
- Wi-Fi content updates deferred until the lesson engine is stable.
- External SD only if hardware inspection confirms a usable slot, or if an SPI SD module is added later.

Avoid excessive flash writes

Live measurements (voltage/current readings during an active lesson) stay in RAM. Only lesson completion, fault summaries, and calibration records get persisted to flash. High-frequency writes of live sensor data would shorten flash life for no benefit.

Lesson engine

The correct architecture is a small lesson engine, not custom firmware per lesson. The engine knows how to display a step, draw a wiring diagram, wait for a knob press, ask a question, read the current sensor, compare expected values against the schema above, and advance.

8. Smart Power Module path

V0: Visible power

Use the Pluggable USB-C meter. The learner reads voltage, current, and watts manually. Sufficient for Ohm's law lessons.

Measurement scope caveat: the Pluggable meter measures the USB-C power path, not necessarily the exact branch current of a breadboard circuit. The meter is excellent for visible V/A/W intuition; the INA219 is the correct path for software-readable, circuit-specific measurement.

V1: Software-observed power

Wire the INA219 in series with the breadboard load, connected to the Elecrow via I2C, following the topology in Figure 3 (§5). The lesson engine reads current and voltage directly.

V1 fault behavior: on over-current or out-of-range read, the lesson engine (1) shows a full-screen red fault banner, (2) displays the explicit instruction "Disconnect power now," and (3) logs the fault event (timestamp and reading, persisted). There is no automatic cutoff at V1 — the learner is the disconnect mechanism. L10 (§11) gives the learner deliberate, supervised practice with this exact banner before they can encounter it by accident.

V2: Protected smart module

Design a custom board with USB-C input, current limiting, measurement, fault reporting, and a safe disconnect circuit. This becomes the commercial product core, and is the first version where "warn" can become "warn and cut."

V3: Row-aware smart breadboard

Only after the power module works, explore sensing rows or continuity. This is much harder and should not be the first product.

9. Kit 1 desk layout

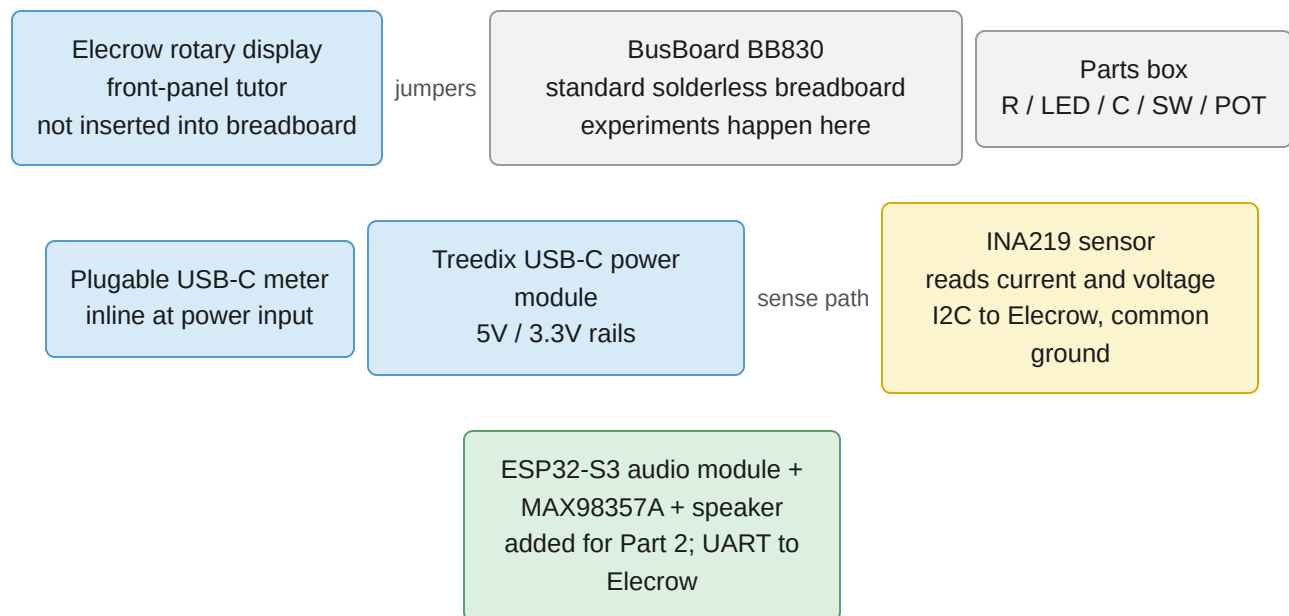


Figure 6. Recommended Kit 1 desk layout (audio module row is new in v1.3).

10. Safety scope

- Kit 1 is low-voltage DC only: 3.3V (default) and 5V (isolated experiments only) breadboard circuits.

- No AC mains. No wall outlet experiments. No exposed lithium battery charging experiments.
- **The Elecrow / ESP32-S3 GPIO and I2C pins are 3.3V logic. Never connect 5V directly to a GPIO, I2C, UART, touch, display, or expansion pin unless the documentation explicitly states 5V tolerance.** This applies equally to the Part 2 audio module's UART link.
- Assume beginner mistakes. Use current limiting, start with higher resistor values, and disconnect power before rewiring — practiced directly in L1 and enforced structurally via `requires_power_off_before_step` (§7).
- The smart module should detect over-current and shut off or warn before damage occurs. At V1, the off-the-shelf module can only warn (§8); L10 gives the learner practiced, supervised exposure to that warning before it can happen unsupervised.
- The audio module and the Elecrow must share a common ground if linked by UART or I2C; the speaker/amp side carries no GPIO-safety concerns of its own since it never touches Elecrow pins directly.

Current limits for early lessons

Lesson	Target current	Note
L2: LED with resistor	< 5 mA at 3.3V	Red/yellow LED, 330Ω–1k resistor.
L3: Resistor comparison	< 10 mA	1k / 470Ω / 330Ω at 3.3V, worst case.
L4/L5: Series & parallel	< 20 mA total	Sum across all branches.
L8: GPIO-driven LED	well under GPIO abs. max	Size the resistor conservatively (target <10 mA) regardless.
L11: Buzzer/transistor	transistor-switched only	Do not drive a buzzer directly from a GPIO pin unless its current draw is verified GPIO-safe.

11. Kit 1 lesson sequence EXTENDED, L12–L13 NEW

L0: First boot validation — before any breadboard wiring: load a vendor example over USB-C and confirm, in order: display output, capacitive touch response, knob rotation, knob press, and a successful USB-C firmware upload. Record the exact board definition, library versions, and example sketch name used.

L1: Power-off habit — the screen asks the learner to disconnect power, then walks through placing/removing a single jumper, then reconnect. No measurement yet.

L2: LED with resistor: Build a simple LED circuit at 3.3V with a red or yellow LED. The screen teaches polarity, current limiting, and expected current (target <5 mA).

L3: Change the resistor: Swap 1k, 470Ω, and 330Ω resistors at 3.3V. The screen asks the learner to predict brightness and current before measuring.

L4: Series resistance: Place two resistors in series and measure lower current.

L5: Parallel branches: Build two LED branches and observe total current increase.

L6: Button input: Wire a pushbutton and pull-down resistor. The display changes when the button is pressed.

L7: Potentiometer: Use a 10K pot as a voltage divider. Turn the pot and watch analog voltage change.

L8: Software-controlled LED: Use the Elecrow to drive an LED through a resistor. Then map knob position to brightness.

L9: Smart measurement: Insert the INA219 into the load path (Figure 3). The display compares expected and measured current.

L10: Controlled fault-screen drill — uses the same safe circuit already on the board, well within the current-limit table. The lesson temporarily lowers its own thresholds far below the circuit's actual safe current, so the existing, unmodified, safe current trips the red fault banner. No real over-current is ever created.

L11: Transistor switch: Use a transistor to drive a buzzer or higher-current LED path from a GPIO pin.

L12: Audio module bring-up NEW — wire the second ESP32-S3 + MAX98357A + speaker on the breadboard, flash a minimal Arduino sketch, and confirm a test tone plays. This is purely a hardware/firmware checkpoint — no Wi-Fi yet.

L13: Internet-radio capstone NEW — connect the audio module to Wi-Fi, fetch a small JSON station list (folding in v1.1/v1.2's old "Stage 2a" Wi-Fi/API teaching), select a saved favorite using the Elecrow knob, and send `play_station` over the UART link. Success is audio from the speaker. This is the Kit 1 finale.

12. Example on-screen lesson flow

1. Screen: Find a 330Ω resistor and a red or yellow LED. Press the knob when found.
2. Screen: Place one end at row A10 and the other at row A15.
3. Screen: Place LED long leg at A15 and short leg to the ground rail.
4. Screen: Disconnect power before connecting the 3.3V rail through the resistor path.
5. Screen: Turn power on. Expected current is about 3 to 5 mA at 3.3V with a red or yellow LED.
6. Screen, V0: Read the Plugable meter and enter or select the closest current.
7. Screen, V1: Sensor reads 3.9 mA. Correct. Press to continue.
8. Screen: Replace the 330Ω resistor with 1k. Predict whether the LED gets brighter or dimmer.

13. Kit 2: Advanced SDR / Waterfall Add-On NEW

Kit 2 is an advanced add-on for users who completed Kit 1, or who already own equivalent components. It carries no beginner electronics curriculum — it is an expansion, not a lesson pack.

Researched compute options

Option	Approx. cost (2026)	Strength	Tradeoff
Raspberry Pi 4	Lower of the two Pi options	Proven for a dedicated single-purpose receiver; huge existing RTL-SDR tutorial base	Less headroom for multiple dongles or heavier processing
Raspberry Pi 5	\$130–205 all-in depending on RAM/storage, due to 2026 DRAM price hikes	Faster CPU and USB; better fit if the station grows (more dongles, OpenWebRX, NVMe)	2026 pricing has risen sharply; no longer a clear price win over a mini PC
x86 mini PC (e.g. Beelink/GMKtec N100/N150 class)	\$170–270	Broad x86 Linux driver/software coverage — fewer "no ARM build exists" dead ends for SDR tools	Less aligned with the maker/tutorial ecosystem the rest of this project lives in

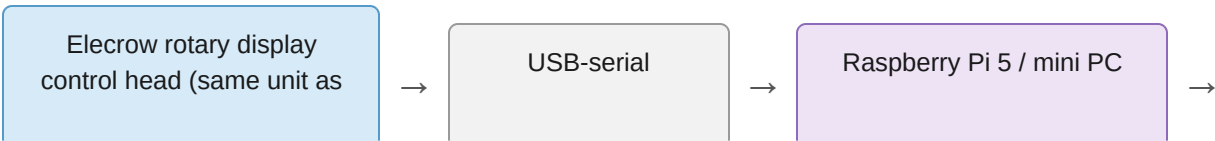
This stays an open call (§1, §4) rather than a hard default: **Raspberry Pi 5 is the recommended starting point** for ecosystem and documentation fit with the rest of this project, with Pi 4 as a cheaper option for a single-purpose receiver and a mini PC as the fallback if Pi pricing or availability is bad at purchase time.

Researched receiver and software

RTL-SDR Blog V4 is the recommended receiver: it replaced the older R820T2 tuner with an R828D tuner plus an internal triplexer, giving native HF reception (500 kHz–1.766 GHz continuous) without an upconverter, and better filtering against FM-broadcast overload than the V3. It is the current default community answer for "what SDR should I buy," at roughly \$35–40.

SDR++ is the recommended first-success waterfall software: it is modern, cross-platform, and runs well on both Pi 4 and Pi 5 with a lighter resource footprint than GQRX. GQRX remains a documented fallback for users who prefer its simpler, more minimal Linux-native interface.

Architecture



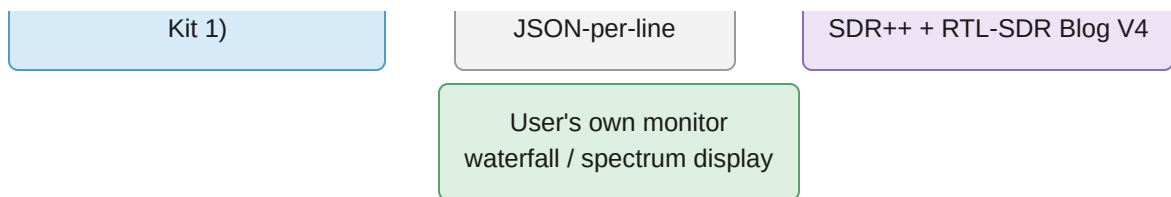


Figure 7. Kit 2 architecture — an add-on to the Kit 1 Elecrow, not a replacement for it.

What Kit 2 assumes from Kit 1, and what it adds

- **Assumes:** the Elecrow unit itself, and that its firmware already emits knob/touch/button events as JSON-per-line over USB-serial (true from Kit 1 Part 1 onward, §6).
- **Adds:** compute (Pi/mini PC), the RTL-SDR receiver, an antenna starter set, USB cables, storage, a power supply, and the SDR++ software stack. The monitor is user-supplied.

Command protocol, generalized further

```
{ "command": "set_frequency", "hz": 145500000 }
{ "command": "set_screen", "mode": "sdr", "frequency_hz": 145500000 }
{ "status_request": "sdr" }
```

14. Kit 3: SDR Workstation with included 15" display NEW

Kit 3 is functionally Kit 2 plus a display: the same SDR/waterfall capability, packaged as a more complete, appliance-like product for users without a spare monitor. Higher cost, higher support burden — this is the premium tier of the ladder, not a different technical path.

Researched display options

Option	Approx. cost	Notes
Official Raspberry Pi Monitor	~\$100	15.6" Full HD IPS, standard HDMI in, USB-powered from the Pi itself. Best documentation/support fit if Kit 2 compute is a Pi.
Budget USB-C/HDMI portable monitor (Arzopa-class)	~\$70	15.6" 1080p, dual USB-C (one supports DisplayPort Alt-Mode) plus mini-HDMI. Works with either a Pi or an x86 mini PC, so it's the right fallback if compute is a mini PC.

Recommendation: the official Raspberry Pi Monitor if Kit 2's compute choice lands on a Pi; an Arzopa-class budget portable monitor if it lands on a mini PC, since the official Pi monitor is purpose-built around Pi power/mounting conventions. Touch is not recommended for v1 — the Elecrow

already provides the physical control surface, and adding touch to the 15" display would duplicate that without a clear benefit.

Architecture

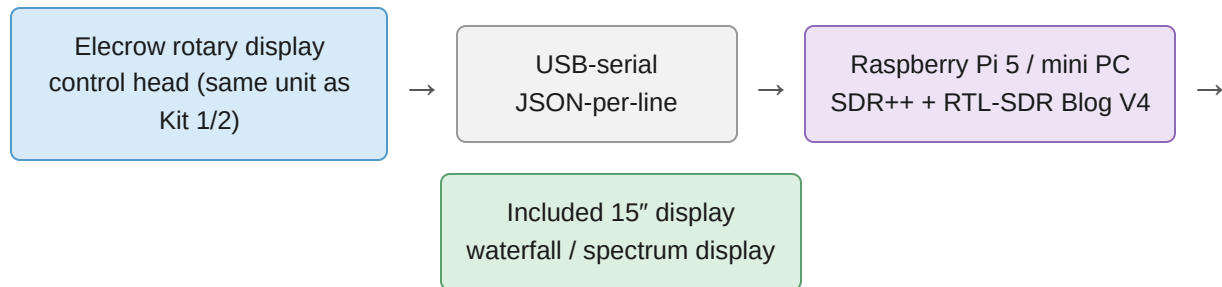


Figure 8. Kit 3 architecture — identical to Kit 2 except the display is bundled, not user-supplied.

15. Bill of materials, by kit and timing RESTRUCTURED

Durable specs live in the Interface/Spec column; ASINs and links are sourcing references only (§18), since listings rot. Items without a confirmed ASIN are described by spec; search by name/part number.

Kit 1 Standard — Buy Now

#	Item	Interface / Spec	Rationale
1	ELECROW ESP32 Rotary Display, 2.1" 480x480 IPS round (already bought)	ESP32-S3, Wi-Fi/BLE, UART, I2C, FPC expansion, 16M flash, 8M PSRAM, 5V/1A input, 3.3V logic on GPIO/I2C	Front-panel tutor and control head for all three kits.
2	BusBoard BB830 solderless breadboard	830 tie points, 4 power rails, 36V/2A rating, 21–26 AWG	Premium standard breadboard, correct size for lessons.
3	Treedix USB-C breadboard power module	USB-C in, 3.3V/5V regulated rail out	3.3V rail is the default for Elecrow-adjacent lessons.
4	Plugable USB-C Power Meter Tester	Inline pass-through, OLED V/A/W readout	Visible Ohm's law (V0). Measures the USB-C input path, not branch current.
5	Adafruit INA219 current sensor, STEMMA QT	High-side DC, 26V target, $\pm 3.2A$, I2C	Smart feedback module (V1). Must sit in series with the load.
6	Qwiic/STEMMA QT cable kit	JST-SH 4-pin to breadboard pins	Confirm Elecrow's own connector before assuming

			Qwiic-compatible (§4).
7	Silicone flexible Dupont jumper wires	M-M, M-F, F-F	Far less stiff than standard kit wires.
8	Aniann 1% metal film resistor assortment	1280 pcs, broad E-series coverage	Needed for every lesson.
9	BOJACK 5mm LED assortment	5 colors	Default lessons use red/yellow at 3.3V.
10	BOJACK tactile pushbutton assortment	10 values	Input lessons.
11	MCIGICM 10K breadboard trim pots	10 pcs, 10K Ω	Voltage dividers, analog input.
12	PLUSIVO BJT transistor assortment	NPN and PNP	L11 transistor switching.
13	Umirokin clear organizer boxes	2 pack, adjustable dividers	Keeps parts from becoming a loose bag.

Kit 1 — Buy Later, General NEW

Not radio-specific. Carried forward from v1.2's "buy soon" rows so a complete Kit 1 box still includes them, and so L11's buzzer has a matching BOM line.

#	Item	Interface / Spec	Rationale
14	EMGTMS ceramic capacitor kit	24 values, 480 pcs, 10pF–10 μ F	RC timing, signal smoothing. Not needed for the first lessons.
15	Hilitchi electrolytic capacitor assortment	300 pcs	Power smoothing for later lessons.
16	Passive/active buzzer module pack	Arduino/ESP32 compatible	L11 (§11) drives this through a transistor, not directly from a GPIO pin, unless the module's current draw is verified GPIO-safe.

Kit 1 Deluxe — Buy Later, for the Radio Capstone

#	Item	Interface / Spec	Rationale
17	ESP32-S3 dev board (second board, audio-dedicated)	ESP32-S3, USB-C, Arduino/PlatformIO compatible	Runs the radio firmware independently of the Elecrow.
18	MAX98357A I2S amplifier breakout	I2S in (BCLK/LRC/DIN, no MCLK), 3W class-D out, 4–8 Ω	Recommended audio path — see §6.

		load	
19	Small 4–8Ω speaker, 3W class	Matched to MAX98357A output	The actual sound source for the capstone.
20	Additional jumper wires / small enclosure (optional)	—	UART link to Elecrow; speaker box is optional polish, not required for the capstone to work.

Kit 2 — Advanced SDR Add-On

#	Item	Interface / Spec	Rationale
21	Raspberry Pi 5 (or Pi 4, or x86 mini PC)	See §13 comparison — open call at purchase time	Heavy compute for FFT/waterfall.
22	RTL-SDR Blog V4 receiver	500 kHz–1.766 GHz, native HF, R828D tuner	Recommended receiver — see §13.
23	Antenna starter set	Telescopic/dipole, SDR-dongle compatible	Minimum viable reception out of the box.
24	USB cables, storage (microSD or SSD), power supply	Matched to compute choice (§13)	Standard Pi/mini-PC accessories.

Kit 3 — SDR Workstation (adds to Kit 2)

#	Item	Interface / Spec	Rationale
25	15" display (official Raspberry Pi Monitor or budget USB-C/HDMI alternative)	HDMI in, USB-powered; see §14 comparison	Bundled screen for an all-in-one workstation.
26	Display power cable, HDMI cable, stand/mount	Matched to display choice	Completes the bundled-display assembly.

16. Product concept

What is distinctive

Most electronics kits give instructions. This system gives feedback: it teaches, observes, and reacts. The device does not merely say what to build. It observes voltage and current and tells the learner what likely happened — and that same teach/observe/react loop is what makes Kit 1's radio capstone land as a "wow" moment rather than a generic feature checkbox: the learner spent Kit 1 Part 1 quietly wiring and being corrected, and the payoff is that the thing they built makes sound.

Why this can be a product

The first product does not require a proprietary breadboard, an SDR receiver, or a 15" screen (§2). The first sellable unit is a smart USB-C power and measurement module, plus an ESP32 display tutor, plus a standard BB830 — Kit 1, Part 1, full stop. Kit 1 Part 2, Kit 2, and Kit 3 are upsells to a working core, not prerequisites for it.

Product realism note: the likely first sellable product is not a smart breadboard, an SDR kit, or a workstation. It is a smart USB-C power and measurement module, paired with a standard BB830, paired with an ESP32 display tutor, with lessons as structured content packs (§7's schema). Kit 1 Part 2's radio capstone is the first upsell; Kit 2 and Kit 3 are further upsells for a smaller, more advanced segment of that same customer base.

Course packs

The same hardware could support course packs: Electronics Fundamentals, Internet Radio, Amateur Radio, SDR Basics, Sensors, Home Automation, and Robotics. With the lesson schema in §7, each course pack is a set of schema files plus assets, not a firmware fork. Internet Radio is now Kit 1's own course pack rather than a Stage 2 appliance feature; SDR Basics becomes Kit 2/3's course pack.

Commercial risk

The difficult parts are not the first hardware. The hard work is curriculum design, safe fault handling, reliable connecting, and making lessons resilient when beginners miswire circuits. The lesson schema and the V1 fault behavior (§7–8) are concrete artifacts against that risk, not just statements of it.

17. Next build action after parts arrive

1. **Photograph and identify the Elecrow connector set** that arrives in the box. Confirm 3.3V, GND, SDA, and SCL pins before connecting the INA219. Add an adapter cable to the BOM if no clean Qwiic/STEMMA-native connector is included (§4).
2. **Run L0's first-boot validation** (§11): load a basic Elecrow example over USB-C, confirm display, touch, knob rotation, knob press, and USB-C upload. Record the board definition, library versions, and example sketch name.
3. Assemble the BB830 rails with the Treedix power module. Verify 5V and 3.3V with the multimeter before connecting the Elecrow.
4. Build the first LED/resistor lesson (L2) manually using the USB-C meter for visible feedback, at 3.3V with a red or yellow LED.

5. Add the INA219 and make the Elecrow display current and voltage, following the topology in Figure 3. That is the first true Smart Lab milestone.
6. **Source and breadboard the ESP32-S3 + MAX98357A audio module** NEW and confirm a test tone plays (L12) before attempting the full internet-radio capstone (L13).
7. **Confirm the UART wiring** between the Elecrow and the audio module, and that both share a common ground.
8. Draft the Kit 2/3 control-message schema extensions (§13) as a one-page note once Kit 1 firmware's event handling is stable, so it does not need a rewrite later.

18. Sourcing references

These are retailer listings or canonical product pages, not the spec of record (see the Interface/Spec columns in §15). Prices, stock, and listing details change; if a link goes stale, search by ASIN, part name, or part number.

- **Elecrow 2.1" rotary display** (ASIN B0G3TZGRC4): rotary knob, capacitive touch, ESP32-S3, Wi-Fi/BLE, UART, I2C, FPC expansion, 16M flash, 8M PSRAM, 5V/1A input. Connector pinout and I2S availability not yet confirmed against the physical board.
- **BusBoard BB830** (ASIN B0040Z4QN8): 830 tie points, 4 power rails, 6.5 x 2.2 x 0.3 in, 36V/2A rating.
- **Treedix USB-C breadboard power module** (ASIN B0CN6HYCJD): USB-C charging/power, 3.3V and 5V outputs.
- **Plugable USB-C power meter** (ASIN B0B5W5NKKN): voltage/ampere measurement, bidirectional connection, data passthrough.
- **Adafruit INA219** (ASIN B09CBSLXN7): high-side current measurement, 26V target voltage, $\pm 3.2\text{A}$ range, I2C address options.
- **ESP32-S3 dev board + MAX98357A I2S breakout** (no single confirmed listing yet): commodity parts from many sellers (e.g. Espressif-licensed ESP32-S3-DevKitC-class boards; MAX98357A breakouts from Adafruit/SparkFun or generic 2-packs). Search by part name — do not assume a specific ASIN until sourced.
- **RTL-SDR Blog V4** (rtl-sdr.com store / official RTL-SDR Blog listing): R828D tuner, native HF via internal triplexer, 500 kHz–1.766 GHz, ~\$35–40.
- **Raspberry Pi 5** (raspberrypi.com): current pricing inflated by 2026 DRAM costs; compare against mini PC options (e.g. Beelink/GMKTec N100/N150 class) at purchase time.
- **Raspberry Pi Monitor, official** (raspberrypi.com/products/raspberry-pi-monitor): 15.6" Full HD IPS, HDMI in, USB-powered. Default Kit 3 display if compute is a Pi.
- **Adafruit INA260** (official page): higher-capacity future option for a custom V2 Smart Power Module, integrated shunt, I2C, 3V/5V logic compatible, up to 36V/15A.

19. Future hardware convergence (v2) NEW

Not a v1.3 dependency: nothing in this section changes or delays the near-term plan. v1.3 stays off-the-shelf and buildable today — Elecrow + breadboard for Kit 1 Standard, plus a second ESP32 + MAX98357A + speaker for Kit 1 Deluxe. This section documents a future direction, gated on proving the MVP first. It is not a design task to start now, and it has no BOM line in §15.

Kit 1 Deluxe currently splits its functions across two boards: the Elecrow (tutor, control head, knob/touch UI) and a second ESP32 audio module (Wi-Fi streaming, I2S decode, amplification). That split is the right v1 architecture — cheap, off-the-shelf, and it keeps each board's firmware simple. It is also, by construction, a preview of a single future device.

Trigger condition

Once the MVP is proven — the lesson engine, the smart measurement loop (INA219 path, §5/§8), and the internet-radio capstone (§6) all working end-to-end on off-the-shelf parts — it becomes worth designing a custom dial/display module that converges what the Elecrow and the second ESP32 audio module do today into one board.

What the custom module should include

- Rotary control (carried from the Elecrow's knob interaction model).
- Touchscreen (carried from the Elecrow's capacitive display).
- A protected expansion connector, so breadboard/sensor wiring mistakes can't easily damage the main board.
- USB-C power and data, for both charging/power and firmware updates.
- A built-in smart-measurement interface (the INA219-class current/voltage sensing path, §5/§8), instead of routing it through an external breakout.
- A clean audio path — an onboard codec, an integrated I2S amplifier, or at minimum a speaker connector — so the radio capstone no longer needs a second physical board.

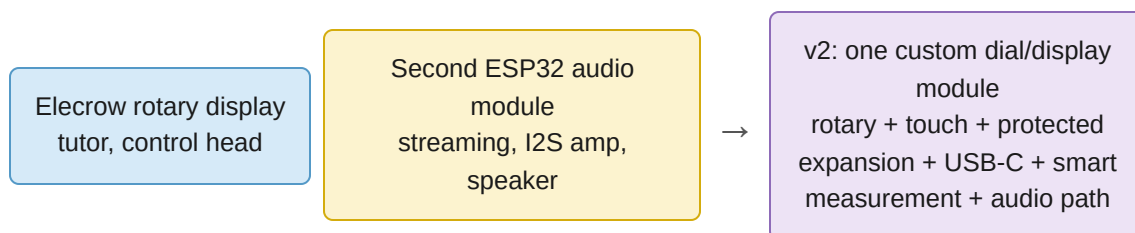


Figure 9. v2 convergence direction — two off-the-shelf boards today, one custom module later, only after the MVP is proven.

This is a hardware roadmap entry, not a current build task: no part numbers, no BOM line, and it should not be started before Kit 1 Standard and Kit 1 Deluxe both work reliably with off-the-shelf parts.

20. User paths: Use Mode, Configure Mode, Developer Mode NEW

The design problem this section answers: the product cannot require the ordinary learner to write code to complete Kit 1 Part 2, Kit 2, or Kit 3. Coding should remain possible, and eventually teachable, but it must not be the default completion path. Without this distinction, the project is an open-source build; with it, the project is a guided learning product.

Hard requirement: a learner must be able to complete the main milestones of Kit 1, Kit 2, and Kit 3 without writing code. The product may teach what the code is doing, and may offer a later "learn to modify the firmware" branch, but the default user should never need to open PlatformIO, edit C++, debug libraries, or compile firmware manually to reach a kit's milestone.

Path A: Use Mode — default, no coding

The user installs prebuilt firmware through a browser-based installer (§22) and follows guided instructions. No step in either flow below requires writing or compiling code.

Kit 1:

1. Plug the Elecrow rotary display into a computer using USB-C.
2. Open the Smart Lab web installer.
3. Click "Install Tutor Firmware."
4. Plug in the second ESP32 audio board (Kit 1 Deluxe only).
5. Click "Install Radio Audio Firmware."
6. Follow the Elecrow screen to configure Wi-Fi, test hardware, build lessons, and play internet-radio audio.

Kit 2 / Kit 3:

1. Flash or boot a prepared Raspberry Pi / mini PC image, or run a packaged installer.
2. Plug in the RTL-SDR receiver.
3. Open the preconfigured SDR application.
4. Confirm the waterfall display.
5. Connect the Elecrow control head.
6. Turn the Elecrow knob and confirm a real SDR parameter changes — frequency, zoom, gain, or preset.

Path B: Configure Mode — no coding, editable settings

Configure Mode lets the user customize the product without programming it. Configurable items include Wi-Fi credentials, the station list, volume defaults, lesson-pack selection, device name,

measurement calibration values, SDR presets, local weather/location settings, the ham repeater database source, and display preferences.

Configuration runs through one or more of: on-device menus, a local web interface served by the device, a JSON/YAML config file (§25), a browser-based setup page, or serial console commands as a fallback only.

Path C: Developer Mode — coding allowed

Developer Mode is for Bob, Claude, Codex, maintainers, and open-source contributors. It uses PlatformIO, the Arduino framework for ESP32 first, the GitHub repository, serial logs, the unit-testable lesson schema files, firmware builds, and release binaries — with ESP-IDF only if later needed. Developer Mode is documented (§21, §23), but it is never required to complete a user-facing kit milestone.

21. Firmware delivery architecture NEW

The product ships prebuilt firmware images so Use Mode (§20) never requires a build step.

Firmware Image 1: Elecrow Tutor Firmware

- Display lesson steps, diagrams, and simple UI.
- Read knob rotation, knob press, and touch input.
- Read the INA219 over I2C and compare measured voltage/current against lesson expectations.
- Display pass / fail / warning states.
- Manage lesson progress.
- Send command messages to the ESP32 audio board (§6) and, later, to the Raspberry Pi/SDR workstation (§13).
- Support Wi-Fi setup, configuration import/export, and version reporting.

Firmware Image 2: ESP32 Audio Firmware

- Connect to Wi-Fi.
- Store or receive station URLs.
- Receive commands from the Elecrow.
- Stream internet radio and decode supported audio formats.
- Output audio over I2S, driving the MAX98357A or equivalent I2S amplifier (§6).
- Report status back to the Elecrow, including human-readable errors.

Expected commands and status replies, extending the protocol already introduced in §6:

```
{"command":"play_station","id":"station_001"}
{"command":"stop"}
{"command":"set_volume","value":42}
{"command":"next_station"}
{"command":"previous_station"}
{"status_request":"audio"}
```

```
{"status":"playing","station":"station_001","buffer_percent":73}
{"status":"stopped"}
{"status":"error","code":"wifi_failed","message":"Wi-Fi connection failed"}
{"status":"error","code":"stream_failed","message":"Could not open stream"}
```

Firmware Image 3: SDR Bridge / Control Head Firmware LATER

This is additional modes enabled in the same Elecrow tutor firmware (Image 1), not a separate codebase — Kit 2/3 reuse the same physical Elecrow unit purchased with Kit 1 (§13).

- Send knob/touch/button events to the Raspberry Pi or mini PC.
- Receive screen-state information from the SDR workstation if needed.
- Display compact SDR status on the Elecrow.
- Select presets; change frequency, waterfall zoom, and gain; toggle modes.

```
{"event":"knob","delta":1}
{"event":"button","name":"select","state":"pressed"}
{"event":"touch","x":214,"y":93}
{"command":"set_frequency","hz":145500000}
{"command":"set_gain","db":22}
{"command":"set_waterfall_zoom","delta":1}
{"status_request":"sdr"}
```

22. Browser-based flashing NEW

The preferred no-code flashing path is a web installer — a tool in the ESP Web Tools / ESP Launchpad class. Arduino IDE and PlatformIO are not required for Use Mode.

Implementation requirements:

- Host the firmware binaries and a manifest/configuration file.
- One install button per supported board.
- Clear board-compatibility display.
- A "known good firmware" fallback build.
- Version numbers and release notes on every build.
- A documented recovery procedure.

- A serial log viewer or simple diagnostic mode, if the chosen web flasher supports it.

Platform caveat: browser-based ESP flashing requires a desktop browser with Web Serial support. iPhone/iPad should not be assumed to work for flashing. Mobile devices are fine for reading instructions, but are not the primary firmware installer.

23. Repository and release structure NEW

Proposed GitHub layout, separating firmware, protocol, lesson content, installers, docs, and hardware so Developer Mode (§20) work stays organized without leaking into the Use Mode experience:

```
smart-electronics-lab/  
  firmware/  
    elecrow-tutor/      platformio.ini, src/, include/, data/, lessons/  
    esp32-audio/        platformio.ini, src/, include/  
  shared-protocol/  
    protocol.md  
    schema/  
  lesson-packs/  
    electronics-fundamentals/ L-1_power_off.yaml, L0_hello_lab.yaml, L1_led_resistor.ya  
    internet-radio/         R1_audio_board_test.yaml, R2_first_station.yaml, assets/  
  installers/  
    web/                  index.html, manifests/, firmware/  
    pi-image/             build-notes.md, install-scripts/, first-boot/  
  docs/  
    product-design/  user-guide/  developer-guide/  troubleshooting/  
  hardware/  
    bom/  wiring-diagrams/  enclosure-notes/
```

Release artifacts: merged Elecrow tutor firmware binary, merged ESP32 audio firmware binary, the web installer manifest, release notes, SHA checksums, compatibility notes, known issues, and recovery instructions.

24. First-time user flow NEW

Written as product behavior, not implementation detail.

Kit 1, Part 1 (Standard): first boot

1. User plugs in the Elecrow.
2. The browser installer flashes Tutor Firmware.

3. The Elecrow boots to "Welcome to Smart Electronics Lab."
4. The device runs input tests: rotate knob, press knob, touch screen, confirm display.
5. The user connects the INA219 and breadboard power path.
6. The tutor displays a wiring diagram.
7. The user builds the LED/resistor lesson (L2, §11).
8. The tutor reads current.
9. The tutor confirms pass, or diagnoses the likely mistake.

Kit 1, Part 2 (Deluxe): radio capstone

1. User plugs in the ESP32 audio board.
2. The browser installer flashes Audio Firmware.
3. The tutor asks the user to connect the audio board to the amplifier/speaker.
4. The tutor sends a test-tone command.
5. The speaker produces a test tone (L12, §11).
6. The user configures Wi-Fi.
7. The user selects the first station.
8. Audio plays.
9. The tutor displays: "You built a networked radio."

This is the core "wow" moment of Kit 1, consistent with §6 and §16.

Kit 2 / Kit 3 first boot

1. User flashes or boots a prepared Pi/mini PC image.
2. User plugs in the RTL-SDR.
3. SDR++ (or the chosen default SDR application) opens with a known first profile.
4. User confirms the waterfall display.
5. User connects the Elecrow.
6. Turning the Elecrow knob changes an SDR parameter.
7. The tutor confirms the control link.

25. Configuration schema NEW

A minimal Kit 1 configuration schema, covering everything Configure Mode (§20) needs to expose without coding:

```
device:
  name: "Smart Lab"
  mode: "kit1"
```

```

wifi:
  setup_mode: "guided"
radio:
  default_volume: 35
  stations:
    - id: "station_001"
      name: "Example Public Radio"
      url: "https://example-stream-url"
      format_hint: "mp3"
lessons:
  active_pack: "electronics-fundamentals"
measurement:
  current_sensor: "ina219"
  current_calibration_mA: 0
safety:
  default_supply_rail: "3v3"
  gpio_logic_voltage: "3v3"
  max_beginner_current_mA: 20

```

Station URLs and format support must be validated by the audio firmware, which returns human-readable errors (§21) when a stream cannot be opened or decoded — not a silent failure.

26. Lesson schema: no-code fields EXTENDED

The §7 lesson schema already carries the safety and measurement-precision fields needed for the fault-drill design. For the no-code product flow, the lesson engine also needs each lesson to self-describe its own firmware and hardware requirements, so Use Mode can validate that a lesson is runnable before starting it:

- `kit`, `part`, `difficulty` — which kit/part this lesson belongs to, and its level.
- `requires_firmware`, `requires_hardware` — minimum firmware version and the hardware parts a lesson needs present.
- `requires_supervision` — carried from §7, surfaced explicitly per lesson.
- `assets`, `hints`, `on_pass`, `on_fail`, `next` — content and branching, already used in §7's example.

```

id: L1_led_with_resistor
title: "LED with resistor"
kit: kit1
part: quiet_lab
difficulty: beginner
requires_firmware: "elecrow-tutor >= 0.1.0"
requires_hardware: [bb830, usb_c_power_module, ina219, red_led, resistor_1k]
supply_rail: 3v3
measurement_source: ina219
expected_units: mA

```

```

expected_range: [1, 4]
max_safe_current_mA: 10
gpio_safe: true
steps:
  - prompt: "Disconnect power before wiring."
    requires_power_off_before_step: true
    wait_for: knob_press
  - prompt: "Place the 1k resistor from row A10 to row A15."
    wait_for: knob_press
  - prompt: "Place the LED long leg at A15 and short leg to ground."
    wait_for: knob_press
  - prompt: "Turn power on."
    measure: current_mA
    expected_range: [1, 4]
    on_fail:
      open_circuit: "No current detected. Check LED polarity and ground."
      over_range: "Current is higher than expected. Check resistor value."
next: L2_change_resistor

```

This is a naming/fields convention, not a second schema: it extends the same lesson-schema family already in production use at §7, §11.

27. Product branch vs. developer branch NEW

Two branches, two audiences:

Product branch

- Prebuilt firmware.
- Browser flashing (§22).
- Guided setup.
- No local development environment.
- No required coding.

Developer branch

- PlatformIO / Arduino.
- Local build and serial debugging.
- Firmware modification.
- Custom lesson packs.
- Protocol extensions.

Hard rule: every required kit milestone must be achievable on the Product branch before it is considered complete. Developer Mode may expose additional learning, but it cannot be the only way to make a kit work.

28. Build plan for Claude/Codex, and the three MVPs NEW

Claude and Codex are the technical implementers; the build order below produces the Product branch (§27) milestones in the order a learner actually needs them, rather than building outward

features first.

1. Create the repository scaffold (§23).
2. Confirm the Elecrow board definition and vendor examples.
3. Build "Hello Lab" Elecrow firmware: display, knob rotation, knob press, touch.
4. Add INA219 readout.
5. Create one hard-coded LED lesson.
6. Convert that lesson to the YAML/structured schema (§26).
7. Build the pass/fail current comparison.
8. Add a web-flashable firmware artifact (§22).
9. Add the ESP32 audio-board proof of concept: Wi-Fi, test tone, stream URL, I2S output to the MAX98357A.
10. Add the Elecrow-to-audio command protocol (§21).
11. Package the first radio-capstone firmware.
12. Only after Kit 1 works, prototype the Kit 2 SDR bridge.

First MVP: a learner follows the Elecrow screen to build one LED/resistor circuit, the INA219 reads current, and the Elecrow gives a correct pass/fail response. (Same MVP as §2.)

Second MVP: the Elecrow sends a play command to a second ESP32 audio board, and audio comes out of a speaker.

Third MVP: the Elecrow knob changes a parameter in an SDR waterfall application running on a Pi or mini PC.

The first MVP is not the entire product (§2). It is the gate every later milestone builds behind.

29. Support, recovery, and flash-storage behavior NEW

Use Mode (§20) must assume some users will miswire hardware, flash the wrong firmware, lose Wi-Fi credentials, or use a power-only USB-C cable. Troubleshooting should be productized, not improvised:

- Known-good firmware recovery and an "erase and reinstall" path.
- Serial log access for troubleshooting, where the web flasher supports it (§22).

- A visible version number on each device.
- Self-test screens: hardware, audio, INA219, and Wi-Fi.
- A "safe mode" boot option, if feasible.

Flash-storage behavior carries forward from §7 unchanged, restated under the no-code framing: live measurements stay in RAM; only lesson completion, summaries, calibration, and settings persist. Lesson data for v1 lives in the internal flash filesystem or is firmware-embedded — no microSD is assumed for the Elecrow. Browser flashing (§22) is the default firmware update path for early product versions; Wi-Fi lesson-pack download remains future work (§7).

30. Kit 2 / Kit 3 image strategy NEW

Extends §13: the no-code user path must use one of a prepared Raspberry Pi image, Raspberry Pi Imager-compatible instructions, a packaged install script, a preconfigured SDR++ profile, or a preinstalled serial bridge service. Kit 2/3 users should not need to compile SDR software.

The Pi/mini PC image should include: SDR++ (or the chosen SDR application, §13), RTL-SDR support, a default waterfall profile, the serial bridge listener for the Elecrow, sample frequency presets, a first-run wizard, and recovery instructions.

31. Product identity statement NEW

Smart Electronics Lab is not a coding kit. It is a guided electronics learning system with an optional developer track. The default learner experience is: install firmware, follow the tutor, build the circuit, observe the measurement, and unlock the next capability. Coding is a later branch, not a prerequisite for success.